National University
of computer and emerging sciences

Final Report

Operating Systems

Semester Project

Name

Syed Inshal Yousaf 22i-7439

Abdul Mueed Malik 22i-1622

Section

OS-B

Submitted to:

Sir Asif Muhammad

Department of Cyber Security BS(CYS)

FAST-NUCES Islamabad

1. Problem Statement

The objective of this project is to simulate the working of a neural network using operating system concepts such as **processes**, **threads**, **mutexes**, **semaphores**, and **inter-process communication (IPC)**. Instead of implementing a neural network in a typical sequential manner, this project distributes computation across layers and neurons using OS-level parallelisms.

The simulation performs the following steps:

1. **Forward Pass:** Input values propagate through the input layer, multiple hidden layers, and the output layer.
2. **Backward Pass:** The output layer computes two values, $f(x_1)$ and $f(x_2)$, which are propagated backward through all layers.
3. **Second Forward Pass:** These computed values are used as fresh inputs and passed forward again through the network.

The system uses **inter-process communication (IPC) via pipes**, and neuron computations rely on data read dynamically from input.txt.

2. System Design & Architecture

2.1 Layer–Process Mapping

Each neural network layer is implemented as a distinct **child process**. The parent process dynamically spawns the required number of layer processes based on user input.

This enables layer-level parallelism and reflects how modern OSes manage distributed workloads.

2.2 Neuron–Thread Mapping

Within each layer process:

- Every neuron is mapped to an independent **pthread thread**.
- All threads execute concurrently to simulate multi-core parallel execution.
- Shared data structures (e.g., layer outputs) are protected by mutexes to avoid race conditions.

2.3 Inter-Process Communication (IPC)

Communication between layers is achieved using **unnamed pipes**, arranged linearly:

Pipe 1: Input Layer → Hidden Layer 1

Pipe 2: Hidden Layer 1 → Hidden Layer 2

Last Pipe: Final Hidden Layer → Output Layer

During the backward pass, the same pipes are reused in reverse direction to propagate the computed values back to previous layers.

2.4 Synchronization

Thread-level synchronization is handled using:

- **Mutex locks** to protect shared arrays

- **Semaphores or controlled execution order** to ensure output aggregation occurs safely
  This prevents data corruption when multiple threads attempt to write simultaneously.

2.5 Data Flow Overview

Forward pass:
input.txt → Input Layer (Process)
→ Hidden Layer 1 (Process)
→ Hidden Layer 2 (Process)
→ ...
→ Output Layer (Process)

Backward:

Output Layer → Hidden Layers → Input Layer

Second forward pass:

f(x1), f(x2) → Input Layer → Hidden Layers → Output Layer

3. Implementation Details

3.1 Input Reading

The file *input.txt* contains:

- Initial input values
- Weight sets for each neuron in each layer
- A structured format allowing dynamic parsing

Each process reads the relevant section of the file based on its assigned layer index. Hardcoding is avoided, allowing flexibility in architecture size.

3.2 Forward Pass Logic

For each neuron thread:

neuron_output = Σ (input[i] × weight[i])

The steps performed by each layer are:

1. Read input vector from its pipe
2. Create *N* threads (one per neuron)
3. Each thread computes its weighted sum
4. Layer aggregates results into an array
5. The array is written into the pipe for the next layer

3.3 Output Layer Computation

After computing total weighted output:

f(x1) = (output² + output + 1) / 2
f(x2) = (output² - output) / 2

These values are:

- Displayed to the console
- Written to *output.txt*
- Sent backward through the pipeline

3.4 Backward Pass Simulation

The backward pass does not adjust weights (no learning).

Instead, it propagates $f(x_1)$ and $f(x_2)$ through pipes in reverse order, allowing each layer to output its received values for verification.

3.5 Second Forward Pass

After the backward phase, the input layer receives $f(x_1)$ and $f(x_2)$ as new inputs and initiates a second forward pass identical to the first, demonstrating multi-phase processing.

3.6 Resource Management

To ensure proper cleanup:

- All pipes are closed using close()
- IPC file descriptors are cleared
- All threads are joined using pthread_join()
- Mutexes and semaphores are destroyed after use
  This ensures no resource leakage or orphaned processes.

4. Sample Outputs

```
mueed@LAPTOP-NM04KFBF:/mnt/c/Users/Abdul Mueed/Desktop/os proj$ ./os
=== Multi-Core Neural Network Simulation ===
Enter number of hidden layers: 2
Enter number of neurons per layer: 2
Input Layer Process Started (PID: 1341)
=== First Forward Pass ===
Hidden Layer 1 Process Started (PID: 1342)
Output Layer Process Started (PID: 1344)
Hidden Layer 2 Process Started (PID: 1343)
Input Layer: Initial inputs: 1.2, 0.5
Layer 0, Neuron 1: Output = 0.56
Layer 0, Neuron 0: Output = 0.22
Hidden Layer 1: Received inputs from previous layer.
Layer 1, Neuron 1: Output = 0.212
Layer 1, Neuron 0: Output = 0.254
Hidden Layer 2: Received inputs from previous layer.
Layer 2, Neuron 0: Output = 0.0678
Layer 2, Neuron 1: Output = 0.161
Output Layer: Received inputs from previous layer.
Layer -1, Neuron 1: Output = 0.06186
Layer -1, Neuron 0: Output = 0.07712
Output Layer: Sum = 0.13898, f(x1) = 0.579148, f(x2) = -0.0598323
Hidden Layer 1: Forward pass completed. Outputs sent to next layer.
Hidden Layer 2: Forward pass completed. Outputs sent to next layer.
Input Layer: Forward pass completed. Outputs sent: 0.22, 0.56
Output Layer: Backward signals sent (f(x1)=0.579148, f(x2)=-0.0598323)
Hidden Layer 2: Received backward signals: 0.579148, -0.0598323
Hidden Layer 1: Received backward signals: 0.579148, -0.0598323
Input Layer: Received backward signals: 0.579148, -0.0598323

=== Second Forward Pass ===
Input Layer: Using backward outputs as new inputs: 0.579148, -0.0598323
Layer 0, Neuron 0: Output = 0.0459483
Layer 0, Neuron 1: Output = 0.149811
Input Layer: Second forward pass completed. Outputs: 0.0459483, 0.149811
```

Code

```
====================================
Hidden Layer 1:
FIRST FORWARD PASS
Hidden Layer 2:
  Neuron 0: 0.254
  Neuron 1: 0.212
Output Layer:
  Neuron 0: 0.07712
====================================
Input Layer:
  Neuron 0: 0.0678
  Neuron 1: 0.161

  Neuron 1: 0.06186
  Sum: 0.13898
  Neuron 0: 0.22
  Neuron 1: 0.56

  f(x1): 0.579148
  f(x2): -0.0598323


====================================
BACKWARD PASS
====================================
Hidden Layer 2 received:
  f(x1): 0.579148
  f(x2): -0.0598323

Hidden Layer 1 received:
  f(x1): 0.579148
  f(x2): -0.0598323

Input Layer received:
  f(x1): 0.579148
  f(x2): -0.0598323
```

Output.txt

6.Work Division

Both members collaborated on the coding and debugging of the project.

- **Documentation & coding:**Mueed
- **Testing and validation:** Inshal

Both contributed equally to designing the architecture and debugging multi-process behavior.

7. Challenges Faced

7.1 IPC Synchronization Issues

Ensuring that reads and writes occurred in the correct order required strict management of pipe endpoints. Incorrect sequencing initially led to blocking and deadlocks.

7.2 Thread Synchronization

Multiple threads writing to shared arrays caused race conditions. Mutex locks were required to ensure atomic updates.

7.3 Parsing input.txt

Mapping weights to the correct layer demanded dynamic parsing logic.
Off-by-one errors and formatting inconsistencies caused incorrect neuron outputs during early testing.

7.4 Process Hierarchy Management

With multiple forked processes, maintaining isolated memory spaces and ensuring correct data flow was challenging. Each process had to manage its own section of the neural pipeline without interfering with the global state.

7.5 Debugging Parallel Execution

Concurrent execution of threads and processes produced nondeterministic output ordering. Diagnostic printing and controlled synchronization were needed to track logic flow accurately.

8. Conclusion

This project successfully demonstrates how operating system concepts can be used to model the behavior of a neural network. By implementing layers as

processes and neurons as threads, the system showcases real parallelism, efficient synchronization, and coordinated IPC mechanisms.

The implementation highlights how OS-level parallelism can be leveraged to simulate neural networks and illustrates the interplay of processes, threads, synchronization, and data flow.